SOLID Principles in React

5 min read Updated 24 Mar 2026 Topic 35 of 62

Bookmark Mark done

6.6 – SOLID Principles in React

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"What are SOLID principles?"	🟡🔴	★★★★☆
"How do you apply SOLID in React?"	🔴	★★★★☆
"Give an example of SRP in React components"	🟡🔴	★★★☆☆

What is SOLID?

SOLID is a set of five design principles from object-oriented programming that help create maintainable, scalable software. While React isn't strictly OOP, these principles map naturally to component-based architecture.

Each component should do one thing well.

A component that fetches data, handles form input, validates, and renders a complex UI is doing too much. Break it into focused parts.

JSX Copy

```jsx
// ❌ One component doing everything
function ProductPage() {
  const [product, setProduct] = useState(null);
  const [reviews, setReviews] = useState([]);
  const [cartCount, setCartCount] = useState(0);
  // ... fetch logic, form handling, validation, rendering all mixed toget
}

// ✅ Each component has one responsibility
function ProductPage() {
  return (
    <div>
      <ProductInfo productId={id} />
      <ProductReviews productId={id} />
      <AddToCartButton productId={id} />
    </div>
  );
}
```

O: Open/Closed Principle

Components should be open for extension but closed for modification.

Instead of modifying a component every time requirements change, design it to accept new behavior through props.

JSX Copy

```
  if (type === 'success') return <div className="green">...</div>;
  if (type === 'error') return <div className="red">...</div>;
  if (type === 'warning') return <div className="yellow">...</div>;
  // Adding a new type means modifying this component
}

// ✅ Open for extension via props
function Alert({ icon, color, children }) {
  return (
    <div style={{ borderColor: color }}>
      {icon && <span>{icon}</span>}
      {children}
    </div>
  );
}

// New variants without touching the Alert component
<Alert color="green" icon="✓">Success!</Alert>
<Alert color="blue" icon="i">Info message</Alert>
```

L: Liskov Substitution Principle

Components that share a common interface should be interchangeable.

If you have a `Button`, a `PrimaryButton`, and a `DangerButton`, any of them should work wherever a button is expected. They should all accept `onClick`, `disabled`, `children`, and other standard button props.

JSX Copy

```
// All button variants accept the same core props
function Button({ children, variant = 'default', ...props }) {
  return <button className={`btn btn-${variant}`} {...props}>{children}</b
}

function PrimaryButton(props) {
  return <Button variant="primary" {...props} />;
```

```
function DangerButton(props) {
  return <Button variant="danger" {...props} />;
}

// Any of these can be used interchangeably
<PrimaryButton onClick={save} disabled={loading}>Save</PrimaryButton>
<DangerButton onClick={remove}>Delete</DangerButton>
```

I: Interface Segregation Principle

Don't force components to accept props they don't use.

Keep props minimal and specific. A component that takes 15 props is hard to understand and easy to misuse.

JSX Copy

```
// ❌ Component receives far more data than it needs
function ProductCard({ product }) {
  // product has: id, name, price, description, images, reviews,
  // inventory, supplier, warehouse, shipping...
  // But this card only shows name and price
  return <div>{product.name} - ${product.price}</div>;
}

// ✅ Only receive what you actually render
function ProductCard({ name, price }) {
  return <div>{name} - ${price}</div>;
}
```

D: Dependency Inversion Principle

Depend on abstractions (props, hooks), not concrete implementations.

Components shouldn't hardcode how they get data. Pass the data or the data-fetching mechanism as a prop, so you can swap implementations easily (especially useful for testing).

JSX Copy

```jsx
// ❌ Hardcoded data fetching
function UserList() {
  const [users, setUsers] = useState([]);
  useEffect(() => {
    fetch('/api/users').then(r => r.json()).then(setUsers);
  }, []);
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

// ✅ Data is passed in, component doesn't care where it came from
function UserList({ users }) {
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

// OR use a custom hook that can be swapped for testing
function UserList() {
  const { users } = useUsers(); // The hook abstracts the data source
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

Quick Reference Table

Principle	React Translation	One-liner
S Single Responsibility	One component = one job	Split big components into fo

Open/Closed	Extend via props, not modification	Use props for new variants, r
L Liskov Substitution	Variants share the same API	PrimaryButton works anywhe
I Interface Segregation	Don't pass unnecessary props	Components receive only wh
D Dependency Inversion	Depend on props/hooks, not hardcoded logic	Pass data in, don't fetch insic

Interview Answer

"SOLID principles from OOP map well to React's component model. Single Responsibility means each component does one thing. Open/Closed means I extend behavior through props instead of modifying components. Liskov Substitution means component variants like PrimaryButton and DangerButton share the same interface and are interchangeable. Interface Segregation means I keep props minimal and don't force components to receive data they don't use. Dependency Inversion means components receive data through props or hooks rather than hardcoding data fetching, which makes them testable and reusable."

Module 6 Complete!

#	Article	Core Concept
6.1	Container/Presentational	Separate data concerns from visual concerns
6.2	Higher-Order Components	Wrap components to add behavior (legacy, replaced by h
6.3	Render Props	Share behavior via function props (legacy, replaced by ho
6.4	Compound Components	Implicit state sharing between related components via Cc
6.5	Provider & Composition	Context-based data sharing, composition over inheritanc
6.6	SOLID Principles in React	Five OOP principles applied to component architecture

Part of the React Interview Preparation Kit — Back to Course Roadmap →

QUICK CHECK — no marks, just for you.

1. A single ProductPage component fetches the product, fetches reviews, handles the add-to-cart form, validates input, and renders the whole layout. According to the lesson, which SOLID principle does this most directly violate, and what is the fix?

- ○ Open/Closed — add a `mode` prop so the component can be extended without being modified

- Liskov Substitution — make ProductPage share the same prop interface as its child components
- Single Responsibility — split it into focused components like ProductInfo, ProductReviews, and AddToCartButton
- Dependency Inversion — wrap the whole page in a Context provider so children read from it

Check answer

2. An Alert component is written as a chain of checks: if type is 'success' render green, if 'error' render red, if 'warning' render yellow. Every new variant means editing Alert. Per the lesson, what does the Open/Closed Principle prescribe here?

- Move the if/else chain into a custom hook so Alert's body stays short
- Design Alert to accept new behavior through props (such as color and icon) so new variants need no edits to Alert
- Keep the if/else chain but replace it with a switch statement for extensibility
- Freeze Alert and create a subclass component for each new variant that inherits its rendering

Check answer

3. ProductCard is given the entire product object — id, name, price, description, images, reviews, inventory, supplier, warehouse, shipping — but it only ever renders the name and the price. Which principle from the lesson does this break?

- Single Responsibility — the component is rendering two separate fields and should be split in two
- Liskov Substitution — ProductCard can no longer be swapped for other card variants that take a product object

Interface Segregation — the component should receive only the props it actually renders, such as name and price

Dependency Inversion — the component depends on a concrete product object instead of fetching the data itself

Check answer

Previous
Provider Pattern & Composition Pattern

Next
Fetching Data: fetch vs axios vs React Query

LEARN MORE

ARTICLES & DOCS

SOLID principles (overview)
wikipedia.org

Thinking in React
react.dev

VIDEOS



React clean code — SOLID examples
YouTube



The right way to write React (SOLID)
YouTube



SOLID for interviews
YouTube

Hand-picked references for this lesson.